CSCI 4311 · Mobile Application Development

Week 5 Lab — Campus Buddy KICT: Add, Star & Enroll

Time: ~90 min · **You need:** Week 4 checkpoint (course list works)

Goal: the list stops being read-only. You will: star a course, add a new course with a popup form, and tick courses you enrolled in — with a live credit-hour total.

> *✓ = what you should see. ★ CHECKPOINT = show your demonstrator.*

A. Move the list into the screen (10 min)

Week 4's `sampleCourses` is `const` — it can never change. Now we want to add and remove courses, so the screen needs its **own copy**.

Step 1. Two additions to `lib/courses_screen.dart`.

First, the imports at the very top of the file. Your Week 4 file has two; add the third:

```
import 'package:flutter/material.dart';
import 'package:campus_buddy_kict/data/courses.dart';
import 'package:campus_buddy_kict/models/course.dart';   // ← NEW this week
```

> *Tip: Why now? Week 4 never wrote the word `Course` in this file — it only used `sampleCourses`, which comes from `data/courses.dart`. This week `List<Course>` names the type, so Dart needs to know which file `Course` lives in. (If you skip this, the error is "Course isn't defined".)*

Second, inside `_CoursesScreenState` add:

```
late List<Course> _courses;

@override
void initState() {
  super.initState();
  _courses = List.of(sampleCourses);   // our own copy, safe to change
}
```

> *Tip: `List.of(...)` makes a **copy**. If you write `_courses = sampleCourses` instead, the app crashes the first time you edit ("unmodifiable list") — you'd be editing the shared const.*

> *Tip: `initState` runs **once**, at the moment the screen is first created — that is exactly why we fill `_courses` there. Remember this for Step 3: after adding or editing `initState`, small `r` (hot reload) is **not** enough — use capital `R` (hot restart), or the app crashes with `LateInitializationError`.*

✓ Nothing changes on screen yet — this step only prepares the ground. Self-check now, because an error here stays silent until Step 5: the `initState` block must be **inside** `_CoursesScreenState` (the private class, under `_courses`), not inside `CoursesScreen` above it.

Step 2. Give the model two new yes/no fields and a copy helper.

This is `lib/models/course.dart` as you have it from Week 4:

```
class Course {
  final String code;
  final String name;
  final int creditHours;
  final String day;
  final IconData icon;
  const Course(this.code, this.name, this.creditHours, this.day, this.icon);
}
```

You change exactly three things — **delete the old one-line constructor and type the class again like this** (lines with `// ←` are new or changed; everything else stays as it is):

```
class Course {
  final String code;
  final String name;
  final int creditHours;
  final String day;
  final IconData icon;
  final bool isFavorite;   // ← NEW: true when the user stars this course
  final bool enrolled;     // ← NEW: true when the user enrolls in it

  // ← CHANGED: constructor now takes two optional extras in {curly braces}
  const Course(this.code, this.name, this.creditHours, this.day, this.icon,
      {this.isFavorite = false, this.enrolled = false});

  // ← NEW method: make a copy with one difference
  Course copyWith({bool? isFavorite, bool? enrolled}) => Course(
        code, name, creditHours, day, icon,
        isFavorite: isFavorite ?? this.isFavorite,
        enrolled: enrolled ?? this.enrolled,
      );
}
```

> *Tip: The three changes in words:* **1)** *two new* `bool` *fields (*`isFavorite`, `enrolled`*);* **2)** *the constructor gets two optional extras inside* `{ }` *— old code like* `Course('CSCI 4311', …)` *still works because they default to* `false`*;* **3)** *one new method* `copyWith`*.*

> *Tip: A* `Course` *can't be changed after it's made (everything is* `final`*). So "change" it = make a* **copy with one difference** *— that's what* `copyWith` *does. It looks odd today, feels normal by Week 10.*

Step 3. Make `build` draw from your `_courses` copy instead of the frozen `sampleCourses`. Four small changes, all inside `build` — each one only renames `sampleCourses` to `_courses`:

Change 1 of 4 — the credit-hours loop at the top of `build`:

```
    for (final c in _courses) {   // ← CHANGED: was sampleCourses
```

Change 2 of 4 — the summary card text:

```
              '${_courses.length} courses · $totalCh credit hours',   // ← CHANGED
```

Change 3 of 4 — the **grid**, two lines:

```
itemCount: _courses.length,   // ← CHANGED
itemBuilder: (context, index) {
  final course = _courses[index];   // ← CHANGED
```

Change 4 of 4 — the **list**, the same two lines again:

```
itemCount: _courses.length,   // ← CHANGED
itemBuilder: (context, index) {
  final course = _courses[index];   // ← CHANGED
```

> *Tip: The one place* `sampleCourses` *survives is inside* `initState` *— that line makes the copy. That is also why the* `data/courses.dart` *import stays.*

Then press `R` — capital R, hot **restart**. Not small `r`: your `initState` was added *after* the app was already running, and hot reload never re-runs `initState` on a State that is already alive. Small `r` here = red screen with `LateInitializationError: Field '_courses' has not been initialized`, even though your code is perfect. `R` rebuilds the app from scratch so `initState` finally runs.

★ CHECKPOINT 1 — looks exactly like Week 4. (Same look, but now it's your list, ready to change.)

B. The star (15 min)

Step 4. In each **list row**, the plain day text on the right becomes a star button, and the day moves into the subtitle.

Your row currently ends like this (from Week 4):

```
subtitle: Text(
  '${course.name} · ${course.creditHours} CH',
),
trailing: Text(course.day),
```

Change it to this (the subtitle text changes; `trailing:` is replaced completely):

```
subtitle: Text(
  '${course.name} · ${course.day}',   // ← CHANGED: day moved in here
),
trailing: IconButton(                 // ← CHANGED: was Text(course.day)
  icon: Icon(
    course.isFavorite
        ? Icons.star
        : Icons.star_border,
    color: course.isFavorite ? Colors.amber : null,
  ),
  onPressed: () => _toggleFavorite(index),
),
```

> *Tip: Read the icon line like a sentence: "if this course is starred, show a filled star; otherwise show only the outline" — and the same question picks gold or no color.*

Step 5. Add the method to the State class:

```
void _toggleFavorite(int index) {
  setState(() {
    _courses[index] =
        _courses[index].copyWith(isFavorite: !_courses[index].isFavorite);
  });
}
```

Read it in slow motion: take the course at `index`, make a copy with the star flipped, put the copy back at `index`, and — because it's inside `setState` — the screen redraws.

Press `r`.

★ CHECKPOINT 2 — tap the star: it fills gold. Scroll far away and back: still gold. (The star lives in the data, not on the screen.)

C. Add Course popup (35 min)

We build the popup the way you should always build something new: the **smallest version that runs first**, then grow it one piece at a time, checking the screen after every piece. Never type 100 lines and hope.

Step 6. Make a new file `lib/add_course_dialog.dart` with the smallest possible popup — a title and one Close button, nothing else:

```
import 'package:flutter/material.dart';

class AddCourseDialog extends StatefulWidget {
  const AddCourseDialog({super.key});

  @override
  State<AddCourseDialog> createState() => _AddCourseDialogState();
}

class _AddCourseDialogState extends State<AddCourseDialog> {
  @override
  Widget build(BuildContext context) {
    return AlertDialog(
      title: const Text('Add a course'),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(context),   // close the popup
          child: const Text('Close'),
        ),
      ],
    );
  }
}
```

> *Tip: Look at the shape: this is a completely normal* `StatefulWidget`*, the same skeleton as your screens since Week 3. A* **dialog is just another widget** *— the popup part is only the* `AlertDialog` *it returns.*

✓ Nothing new appears on screen yet — no button opens it. That is the next step. But VS Code shows no red underlines in the new file.

Step 7. Open it.

First, `courses_screen.dart` needs to know the new file exists — add its import at the top, under the other two:

```
import 'package:flutter/material.dart';
import 'package:campus_buddy_kict/data/courses.dart';
import 'package:campus_buddy_kict/models/course.dart';
import 'package:campus_buddy_kict/add_course_dialog.dart';   // ← NEW this step
```

> *Tip: Same rule as Step 1: using a name that lives in another file (*`AddCourseDialog`*) means importing that file. Skip it and the error is "AddCourseDialog isn't defined".*

Then, in `_CoursesScreenState`, add this method under `_toggleFavorite`:

```
  void _openAddDialog() {              // ← NEW
    showDialog(
      context: context,                // open it on top of this screen
      builder: (_) => const AddCourseDialog(),
    );
  }
```

and connect it to a **+** button in the top bar. Your `actions:` list currently has one button (the grid toggle) — add the new one in front of it:

```
      actions: [
        IconButton(                          // ← NEW: opens the Add popup
          icon: const Icon(Icons.add),
          onPressed: _openAddDialog,
        ),
        IconButton(
          icon: _isGrid ? const Icon(Icons.grid_on) : const Icon(Icons.list),
          tooltip: 'Grid view',
          onPressed: () => setState(() => _isGrid = !_isGrid),
        ),
      ],
```

> *Tip:* `showDialog` *= "open a popup on top of this screen."* `builder:` *is the function Flutter calls to draw it — you know this slot already: it is the same "give me a function, I call it later" idea as* `onPressed`*.*

✓ Press `r`, tap **+** → the popup appears with its title; Close dismisses it. The hard part (opening a popup) already works — everything from here on just adds fields to it.

Step 8. Give the dialog a memory, and its first field.

Add two fields at the top of `_AddCourseDialogState`, and a `dispose` method (you met controllers and `dispose` in Week 3):

```
class _AddCourseDialogState extends State<AddCourseDialog> {
  final _formKey = GlobalKey<FormState>();      // ← NEW: the form's name tag
  final _codeCtrl = TextEditingController();    // ← NEW: remembers the typing

  @override
  void dispose() {                              // ← NEW: clean up on close
    _codeCtrl.dispose();
    super.dispose();
  }
```

Then put one text field inside the popup. Your `build` currently returns `AlertDialog(title: …, actions: …)`; the field goes between them, wrapped in a `Form`:

```
    return AlertDialog(
      title: const Text('Add a course'),
      content: Form(                            // ← NEW: groups the fields
        key: _formKey,                          // ← NEW: the tag from above
        child: TextFormField(                   // ← NEW: the first field
          controller: _codeCtrl,
          decoration: const InputDecoration(
            labelText: 'Code (e.g. CSCI 4311)',
          ),
        ),
      ),
      actions: [                                // unchanged
        TextButton(
          onPressed: () => Navigator.pop(context),
          child: const Text('Close'),
        ),
      ],
    );
```

✓ Press `r`, tap **+** → the popup now has a text field you can type in. It does not check anything yet.

Step 9. Teach the field to check itself.

Add a `validator` to the field (a function that returns an error message, or `null` when the input is fine):

```
        child: TextFormField(
          controller: _codeCtrl,
          decoration: const InputDecoration(
            labelText: 'Code (e.g. CSCI 4311)',
          ),
          validator: (v) {                          // ← NEW from here
            final t = v?.trim() ?? '';
            if (t.isEmpty) return 'Code is required';
            if (!RegExp(r'^[A-Z]{4}\s?\d{4}$').hasMatch(t)) {
              return 'Format: 4 letters + 4 digits';
            }
            return null;   // returning null means "this input is fine"
          },
        ),
```

> *Tip: The* `RegExp(r'^[A-Z]{4}\s?\d{4}$')` *line is a* **pattern** *read like: "exactly 4 capital letters, then an optional space, then exactly 4 digits" —* `CSCI 4311` *fits,* `hello` *does not.*

Validators are lazy — they only run when something calls `validate()`. That caller is the **Add** button. Add the method:

```
void _submit() {   // ← NEW
  if (!_formKey.currentState!.validate()) return;   // run every check
  Navigator.pop(context);   // for now: a valid form just closes
}
```

and give the popup its second button (and rename Close → Cancel, since "cancel" is what it now means):

```
    actions: [
      TextButton(
        onPressed: () => Navigator.pop(context),
        child: const Text('Cancel'),           // ← CHANGED: was 'Close'
      ),
      FilledButton(                            // ← NEW: the check-and-add button
        onPressed: _submit,
        child: const Text('Add'),
      ),
    ],
```

✓ Press `r`. Add with an **empty** code → red "Code is required", the popup stays open. Type `hello` → red format message. Type `CSCI 4311` → the popup closes.

Step 10. Second field: the course name.

Add one more controller (and its cleanup line):

```
final _nameCtrl = TextEditingController();   // ← NEW
```

```
@override
void dispose() {
  _codeCtrl.dispose();
  _nameCtrl.dispose();                       // ← NEW
  super.dispose();
}
```

Now there are two fields, so the `Form`'s child becomes a `Column` (a vertical list — Week 2) inside a `SingleChildScrollView` (so the keyboard can never squeeze the fields off-screen):

```
    content: Form(
      key: _formKey,
      child: SingleChildScrollView(          // ← NEW
        child: Column(                       // ← NEW: one field per row
          mainAxisSize: MainAxisSize.min,
          children: [
            TextFormField(                   // the code field, unchanged
              controller: _codeCtrl,
              decoration: const InputDecoration(
                labelText: 'Code (e.g. CSCI 4311)',
```

```
              ),
              validator: (v) {
                final t = v?.trim() ?? '';
                if (t.isEmpty) return 'Code is required';
                if (!RegExp(r'^[A-Z]{4}\s?\d{4}$').hasMatch(t)) {
                  return 'Format: 4 letters + 4 digits';
                }
                return null;
              },
            ),
            TextFormField(                    // ← NEW: the name field
              controller: _nameCtrl,
              decoration:
                  const InputDecoration(labelText: 'Course name'),
              validator: (v) => (v?.trim().isEmpty ?? true)
                  ? 'Name is required'
                  : null,
            ),
          ],
        ),
      ),
    ),
```

✓ Press `r` — Add with an empty name → its own red message, even when the code is fine. Each field checks itself separately.

Step 11. Third input: the day, as a dropdown.

Add the data for the dropdown at the top of the State class:

```
String _day = 'Monday';                          // ← NEW: current choice
static const _days = [                           // ← NEW: the options
  'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday'
];
```

and add the dropdown as the Column's third child (after the name field):

```
            DropdownButtonFormField<String>(     // ← NEW
              initialValue: _day,
              items: _days
                  .map((d) => DropdownMenuItem(value: d, child: Text(d)))
                  .toList(),
              onChanged: (v) => setState(() => _day = v!),
            ),
```

> *Tip:* `_days.map(...)` *reads: "for every day in the list, make one menu item."* `v!` *is the same not-null promise as in the handout's radio example.*

✓ Press `r` — the popup now offers the five days; picking one stores it in `_day`.

Step 12. Last input: credit hours, as a slider.

Add the number it stores:

```
int _creditHours = 3;   // ← NEW: the slider's current value
```

and add this as the Column's fourth child (after the dropdown):

```
            const SizedBox(height: 8),           // ← NEW: a little air
            Row(                                 // ← NEW: label + slider + number
              children: [
                const Text('Credit hours'),
                Expanded(
                  child: Slider(
                    min: 1, max: 4, divisions: 3,
                    label: '$_creditHours',
                    value: _creditHours.toDouble(),
                    onChanged: (v) =>
                        setState(() => _creditHours = v.round()),
                  ),
                ),
                Text('$_creditHours'),
              ],
            ),
```

> *Tip: The slider speaks in `double`s, we count in whole numbers — `.toDouble()` and `.round()` translate between the two, exactly like the handout promised.*

✓ Press `r` — drag the slider: it snaps to 1, 2, 3, 4 and the number beside it follows live.

Step 13. Make the popup hand back a `Course`.

The dialog needs the model now — add the import at the top of the file (under the Flutter one):

```
import 'package:campus_buddy_kict/models/course.dart';   // ← NEW
```

Then `_submit` grows: instead of just closing, it packs the answers into a new `Course` and closes **with it as the answer**:

```
  void _submit() {
    if (!_formKey.currentState!.validate()) return;
    final course = Course(                  // ← NEW: pack the answers
      _codeCtrl.text.trim().toUpperCase(),
      _nameCtrl.text.trim(),
      _creditHours,
      _day,
      Icons.auto_stories,
    );
    Navigator.pop(context, course);         // ← CHANGED: close AND answer
  }
```

> *Tip: `Navigator.pop(context, course)` = "close the popup, and `course` is my answer." Cancel still pops with no value — its answer is `null`. This is the "screen answers back" idea from the handout, and it becomes big in Week 6.*

Step 14. The list must receive the answer. Back in `courses_screen.dart`, grow `_openAddDialog` into an async method that waits for the popup's answer and adds it to the list:

```
Future<void> _addCourse() async {        // ← CHANGED: was _openAddDialog
  final course = await showDialog<Course>(
    context: context,
    builder: (_) => const AddCourseDialog(),
  );
  if (course == null) return;   // the user cancelled, so do nothing
  setState(() => _courses.add(course));
}
```

and the button now calls the new name (one word changes):

```
        IconButton(
          icon: const Icon(Icons.add),
          onPressed: _addCourse,         // ← CHANGED: was _openAddDialog
        ),
```

> *Tip:* `await` *= "pause this function until the popup answers." And* `setState(() => _courses.add(course))` *is the loop from the handout: the data changes, so the screen redraws — with one row more.*

★ CHECKPOINT 3 — press +: popup opens. Try saving with an empty form → red errors, no save. Fill it properly → new row appears in the list, the star works on it, and the summary card counts it.

D. Enroll + live totals (15 min)

Step 15. Make tapping a row toggle "enrolled" — the same trick as the star, but with the `enrolled` field.

First, add this method directly under `_toggleFavorite`:

```
void _toggleEnrolled(int index) {   // ← NEW: same shape as the star
  setState(() {
    _courses[index] = _courses[index].copyWith(
      enrolled: !_courses[index].enrolled,
    );
  });
}
```

Second, the row's `onTap` currently shows a SnackBar (from Week 4). It becomes a call to the new method:

```
                    onTap: () => _toggleEnrolled(index),   // ← CHANGED
```

(you are deleting the three `ScaffoldMessenger` lines and typing this one line instead).

Step 16. The summary card shows a second number — the credit hours you actually *enrolled* in.

First, add this at the top of `build`, directly under the `totalCh` loop:

```
    final enrolledCh = _courses                // ← NEW
        .where((c) => c.enrolled)
        .fold<int>(0, (sum, c) => sum + c.creditHours);
```

> *Tip: Read it like a sentence:* `where` *keeps only the courses you are enrolled in, and* `fold` *walks that short list adding up each one's credit hours, starting from 0.*

Second, the card text gains a second line:

```
                '${_courses.length} courses · $totalCh credit hours\n'
                'Enrolled: $enrolledCh CH',    // ← NEW: the second line
```

> *Tip: Dart joins string pieces that sit next to each other, and* `\n` *means "start a new line here".*

Step 17. Show enrolled rows differently: the little icon on the left becomes a green tick when you are enrolled in that course.

Your leading icon is currently:

```
                        child: Icon(
                          course.icon,
                          color: Theme.of(context)
                              .colorScheme
                              .onPrimaryContainer,
                        ),
```

Change it to:

```
                        child: Icon(
                          course.enrolled            // ← CHANGED: was course.icon
                              ? Icons.check_circle
                              : course.icon,
                          color: course.enrolled     // ← CHANGED
                              ? Colors.green
                              : Theme.of(context).colorScheme.onPrimaryContainer,
                        ),
```

★ CHECKPOINT 4 — tap a row: green tick appears and the enrolled total changes in the same instant.

E. Save your week (3 min)

```
git add .
git commit -m "week 5 add star enroll"
```

`.gitignore` **this week: nothing to change.** One file was added (`add_course_dialog.dart`) — source code, git saves it.

Exercise in class (hand in before leaving)

Duplicate guard. The popup currently lets you add `CSCI 4311` twice. Fix: pass the existing codes into the dialog — change its constructor to `AddCourseDialog({super.key, required this.existingCodes})` with a `final List<String> existingCodes;` field, call it with `AddCourseDialog(existingCodes: _courses.map((c) => c.code).toList())`, and add to the code validator: if the code is in the list, return `'Already added'`. Test it.

Homework (Assignment 3, graded — finishes last week's prep)

1. If the Week 4 day filter broke, fix it — it must work on `_courses`. 2. Add **remove**: long-press a row → "Are you sure?" dialog → remove the course inside `setState`. (Hint: `onLongPress:`, `showDialog` with two buttons, pop with `true`.) 3. Add a `FilterChip` row: "★ favourites only" — when on, the list shows only starred courses. Same `_courses`, just a filtered view.

If something goes wrong

Problem	Fix
`LateInitializationError: Field '_courses' has not been initialized`	Two possible causes: the `initState` half of Step 1 is missing (or sits in the wrong class), or you pressed small `r` instead of capital `R` — hot reload never re-runs `initState`. Add/fix `initState`, then hot-restart with `R`.
Star works, but courses added later never appear	Build still reads `sampleCourses` — do Step 3: every `sampleCourses` in `build` becomes `_courses` (the `initState` copy line keeps `sampleCourses`).
"Cannot modify an unmodifiable list"	You wrote `_courses = sampleCourses` — use `List.of(sampleCourses)`
Validators never show	The button must call `_formKey.currentState!.validate()`
Keyboard covers the popup fields	`SingleChildScrollView` inside the dialog (already in the code — keep it)
New course loses its star later	Somewhere you rebuilt `_courses` from `sampleCourses` again — copy once, in `initState` only
Slider complains about types	Write `min: 1, max: 4` like the code above; `value:` must be `.toDouble()`
`Bad state: no element`	You used `firstWhere` on an empty list — use `.where(...).toList()`

Next lab:

TEST 1 in the first hour, then the app grows menus and pages — a Home screen, bottom navigation, a real course detail page.